



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251187
Nama Lengkap	Benedictina Andhika Chinor Jodie Soesila
Minggu ke / Materi	13 / Tuple

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Tuple Immutable

Tuple bisa menyerupai list, tetapi bersifat immutable, artinya isi atau elemennya tidak dapat diubah setelah dibuat. Tuple dapat menyimpan berbagai tipe data dan menggunakan indeks integer untuk mengakses elemennya. Karena bersifat immutable dan hashable, tuple dapat digunakan sebagai key pada dictionary.

Tuple dapat ditulis tanpa atau dengan tanda kurung:

```
t = 'a', 'b', 'c', 'd', 'e'
t = ('a', 'b', 'c', 'd', 'e')
```

Jika tuple hanya memiliki satu elemen, harus ditambahkan tanda koma agar dikenali sebagai tuple:

```
t1 = ('a',)
```

Tuple juga dapat dibuat menggunakan fungsi tuple().

```
t = tuple('dutawacana')
print(t)
```

Elemen tuple dapat diakses menggunakan indeks dan slicing:

```
t = ('a', 'b', 'c', 'd', 'e')

print(t[0])
print(t[1:3])
```

Karena tuple immutable, elemennya tidak bisa diubah langsung:

```
t[0] = 'A'
```

Kode tersebut akan menghasilkan error. Namun, tuple masih bisa “diganti” dengan membuat tuple baru, misalnya:

```
t = ('A',) + t[1:]
print(t)
```

Program tersebut menghasilkan tuple baru dengan elemen pertama diganti menjadi 'A'.

Membandingkan Tuple

Operator perbandingan pada tuple bekerja dengan membandingkan elemen pertama dari setiap tuple. Jika elemen pertama sama, maka Python akan membandingkan elemen berikutnya hingga ditemukan perbedaan.

```
(0, 1, 2) < (0, 3, 4)
True
```

```
(0, 1, 2000000) < (0, 3, 4)
True
```

Fungsi `sort()` pada Python juga bekerja dengan cara yang sama, yaitu mengurutkan berdasarkan elemen pertama, lalu elemen berikutnya jika diperlukan. Teknik ini dikenal sebagai DSU (Decorate, Sort, Undecorate), yaitu membuat tuple sebagai key pengurutan, melakukan sorting, lalu mengambil kembali data yang sudah diurutkan.

Contoh program berikut digunakan untuk mengurutkan kata dalam kalimat berdasarkan panjang kata dari yang paling panjang ke paling pendek:

```
kalimat = 'but soft what light in yonder window breaks'

dafkata = kalimat.split()
t = list()

for kata in dafkata:
    t.append((len(kata), kata))

t.sort(reverse=True)

urutan = list()

for length, kata in t:
    urutan.append(kata)

print(urutan)
```

Pada program tersebut, setiap kata disimpan dalam bentuk tuple (panjang_kata, kata). Fungsi `sort(reverse=True)` digunakan untuk mengurutkan data secara descending berdasarkan panjang kata. Setelah diurutkan, kata-kata dimasukkan kembali ke list urutan sehingga menghasilkan daftar kata dari yang paling panjang ke yang paling pendek.

Penugasan Tuple

Python memiliki fitur tuple assignment, yaitu kemampuan menggunakan tuple di sisi kiri proses penugasan sehingga beberapa variabel dapat diisi sekaligus dalam satu statement. Contohnya:

```
m = ['have', 'fun']
x, y = m

print(x)
print(y)
```

Kode tersebut akan mengisi x dengan elemen pertama dan y dengan elemen kedua dari list m.

Penulisan ini sama seperti:

```
x = m[0]
y = m[1]
```

Tuple assignment juga dapat ditulis menggunakan tanda kurung:

```
(x, y) = m
```

Selain itu, fitur ini sering digunakan untuk menukar nilai variabel dengan lebih singkat:

```
a, b = b, a
```

Pada tuple assignment, jumlah variabel di sebelah kiri harus sama dengan jumlah nilai di sebelah kanan. Jika tidak sama, akan muncul error:

```
a, b = 1, 2, 3
```

Tuple assignment juga dapat digunakan pada hasil fungsi seperti `split()`. Contohnya untuk memisahkan username dan domain email:

```
email = 'didanendya@ti.ukdw.ac.id'

username, domain = email.split('@')

print(username)
print(domain)
```

Dictionaries and Tuple

Dictionary memiliki method `items()` yang digunakan untuk mengambil seluruh pasangan **key-value** dalam bentuk list berisi tuple. Setiap tuple terdiri dari key dan value dari dictionary.

```
d = {'a':10, 'b':1, 'c':22}

t = list(d.items())

print(t)
```

Hasilnya berupa list tuple seperti:

```
[('b', 1), ('a', 10), ('c', 22)]
```

Karena hasil `items()` berbentuk list, data tersebut dapat diurutkan menggunakan `sort()`. Proses sorting akan mengurutkan tuple berdasarkan elemen pertama, yaitu key.

```
t.sort()

print(t)
```

Hasil akhirnya menjadi:

```
[('a', 10), ('b', 1), ('c', 22)]
```

Multipenugasan dengan dictionaries

Kombinasi `items()`, tuple assignment, dan `for` memungkinkan key dan value dictionary diproses sekaligus dalam satu loop.

```
for key, val in d.items():
    print(val, key)
```

Karena `items()` menghasilkan pasangan (key, value), maka kedua data dapat langsung disimpan ke variabel `key` dan `val`.

Dictionary juga dapat diurutkan berdasarkan value dengan membuat tuple (value, key) lalu menggunakan `sort()`.

```
d = {'a':10, 'b':1, 'c':22}

l = list()

for key, val in d.items():
    l.append((val, key))

l.sort(reverse=True)

print(l)
```

Kata yang sering muncul

Program ini digunakan untuk menghitung frekuensi kemunculan kata pada file `romeo-full.txt` dan menampilkan 10 kata yang paling sering muncul. File dibaca baris per baris, lalu tanda baca dihapus menggunakan `translate()`, semua huruf diubah menjadi lowercase dengan `lower()`, dan kata dipisahkan menggunakan `split()`. Setiap kata kemudian dihitung menggunakan dictionary `counts`.

```
if word not in counts:
    counts[word] = 1
else:
    counts[word] += 1
```

Setelah proses perhitungan selesai, isi dictionary diubah menjadi list tuple (value, key) agar dapat diurutkan berdasarkan jumlah kemunculan kata menggunakan `sort(reverse=True)`.

```
for key, val in counts.items():
    lst.append((val, key))
```

Terakhir, program menampilkan 10 kata dengan frekuensi tertinggi menggunakan slicing `lst[:10]`.

```
for key, val in lst[:10]:
    print(key, val)
```

Program ini menunjukkan bagaimana Python dapat digunakan untuk analisis data teks dengan memanfaatkan dictionary, tuple, list, dan sorting.

Tuple sebagai kunci dictionaries

Tuple bersifat hashable sehingga dapat digunakan sebagai key pada dictionary, sedangkan list tidak bisa digunakan karena bersifat mutable. Tuple sering digunakan sebagai composite key, yaitu key yang terdiri dari lebih dari satu nilai.

Contohnya pada program direktori telepon berikut:

```
last = 'nendya'
first = 'dida'
number = '088112266'

directory = dict()

directory[last, first] = number
```

Pada kode tersebut, (last, first) otomatis dianggap sebagai tuple dan digunakan sebagai key untuk menyimpan nomor telepon pada dictionary.

Untuk menampilkan isi dictionary digunakan looping berikut:

```
for last, first in directory:
    print(first, last, directory[last, first])
```

Tuple key akan dipecah ke variabel last dan first menggunakan tuple assignment, lalu program mencetak nama dan nomor telepon yang sesuai.

BAGIAN 2: LATIHAN MANDIRI (60%)

Link Github: https://github.com/chinorjodie/71251187_chinor

SOAL 1

Source Code:

```
def sama(tA):  
    n = set(tA)  
    if len(n) == 1:  
        return True  
    else:  
        return False  
  
tA = eval(input("tA= "))  
print(sama(tA))
```

Output:

```
PS D:\71251187_chinor\minggu13> py .\soal01.py  
tA= 10,20,30,40,10  
False  
PS D:\71251187_chinor\minggu13> py .\soal01.py  
tA= 10,10,10,10,10,10,10,10  
True  
PS D:\71251187_chinor\minggu13> █
```

Penjelasan:

Program ini digunakan untuk memeriksa apakah semua elemen dalam sebuah tuple memiliki nilai yang sama atau tidak. Fungsi **sama(tA)** menerima parameter berupa tuple, lalu mengubah isi tuple tersebut menjadi Set dengan **set(tA)**. Karena Set hanya menyimpan nilai unik, jika seluruh elemen pada tuple sama maka jumlah elemen pada Set hanya satu. Program kemudian mengecek kondisi **len(n) == 1**; jika benar maka fungsi mengembalikan nilai **True**, sedangkan jika terdapat lebih dari satu nilai unik maka fungsi mengembalikan **False**. Setelah itu, program

meminta input tuple dari user menggunakan `eval(input())`, kemudian hasil pengecekan ditampilkan dengan `print(sama(tA))`.

SOAL 2

Source Code:

```
def data(dataku):
    nim = dataku[1]
    nama = dataku[0]
    alamat = dataku[2]

    print(f"NIM      : {nim}")
    print(f>Nama      : {nama}")
    print(f"Alamat   : {alamat}\n")

    ambil = nama.split()
    nama_depan = ambil[0]
    print(f"NIM: {tuple(nim)}\n")
    print(f"NAMA DEPAN: {tuple(nama_depan[1:])}\n")
    print(f"NAMA TERBALIK: {tuple(ambil[::-1])}")

datacinoy = ("Benedictina Andhika Chinor Jodie Soesila", "71251187",
"Suryowijayan, Mantrijeron")
data(datacinoy)
```

Output:

```

PS D:\71251187_chinor\minggu13> py .\soal02.py
NIM      : 71251187
Nama     : Benedictina Andhika Chinor Jodie Soesila
Alamat   : Suryowijayan, Mantrijeron

NIM: ('7', '1', '2', '5', '1', '1', '8', '7')

NAMA DEPAN: ('e', 'n', 'e', 'd', 'i', 'c', 't', 'i', 'n', 'a')

NAMA TERBALIK: ('Soesila', 'Jodie', 'Chinor', 'Andhika', 'Benedictina')
PS D:\71251187_chinor\minggu13> 

```

Penjelasan:

Program ini membuat fungsi **data(dataku)** yang digunakan untuk mengambil dan menampilkan informasi dari sebuah tuple. Data pada tuple disimpan ke dalam variabel **nim**, **nama**, dan **alamat** menggunakan indeks tuple, yaitu **dataku[1]**, **dataku[0]**, dan **dataku[2]**. Setelah itu, data ditampilkan menggunakan **print()** dan format string **f""** agar output lebih rapi. Bagian **nama.split()** digunakan untuk memisahkan nama lengkap menjadi beberapa kata dan hasilnya disimpan ke variabel **ambil**. Kemudian, **ambil[0]** mengambil nama depan dan disimpan ke variabel **nama_depan**. Program juga menggunakan **tuple(nim)** untuk mengubah setiap karakter pada NIM menjadi tuple per karakter. Selanjutnya, **tuple(nama_depan[1:])** mengambil huruf nama depan mulai dari indeks ke-1 hingga akhir lalu mengubahnya menjadi tuple. Pada bagian terakhir, **ambil[::-1]** digunakan untuk membalik urutan nama, kemudian diubah menjadi tuple dengan **tuple()**. Terakhir, tuple **datacinoy** yang berisi nama, NIM, dan alamat dimasukkan ke fungsi **data(datacinoy)** untuk diproses dan ditampilkan hasilnya.

SOAL 3

Source Code:

```

import re
fname = input("Enter a file name: ")
fopen = open(fname)

```

```

hitung = {}

for i in fopen:
    if not i.startswith("From "):
        continue
    ambil = re.findall(r"(\d{2}):\d{2}:\d{2}", i)
    jam = ambil[0]
    if jam not in hitung:
        hitung[jam] = 1
    else:
        hitung[jam] += 1

for jam in sorted(hitung):
    print(jam, hitung[jam])

```

Output:

```

PS D:\71251187_chinor\minggu13> python -u "d:\71251187_chinor\minggu13\soal03.py"
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
PS D:\71251187_chinor\minggu13>

```

Penjelasan:

Program ini digunakan untuk menghitung jumlah kemunculan jam pengiriman email dari sebuah file. Pertama, program mengimpor modul **import re** untuk menggunakan regular expression. Lalu user diminta memasukkan nama file melalui **input("Enter a file name: ")**, kemudian file dibuka menggunakan **open(fname)** dan disimpan ke variabel **fopen**. Dictionary kosong **hitung = {}**

digunakan untuk menyimpan jumlah kemunculan setiap jam. Pada bagian **for i in fopen:**, program membaca file baris per baris. Kondisi **if not i.startswith("From ")**: **continue** digunakan untuk melewati baris yang tidak diawali kata "From ". Setelah itu, **re.findall(r"(\d{2}):\d{2}:\d{2}", i)** digunakan untuk mengambil bagian jam dari format waktu seperti 09:14:16, sehingga yang diambil hanya angka jamnya saja, misalnya 09. Hasilnya disimpan ke variabel **jam = ambil[0]**. Program kemudian menghitung jumlah kemunculan setiap jam menggunakan dictionary dengan kondisi **if jam not in hitung**:. Jika jam belum ada di dictionary maka dibuat dengan nilai awal 1, sedangkan jika sudah ada maka nilainya ditambah menggunakan **hitung[jam] += 1**. Terakhir, **sorted(hitung)** digunakan untuk mengurutkan jam secara ascending sebelum ditampilkan menggunakan **print(jam, hitung[jam])**.